ImageLab v2 Report

CMPS4191: Advanced Web Technologies
University of Belize
Andres Hung & Jennessa Sierra
Oct 27, 2026

GitHub Repository: https://github.com/andreshungbz/imagelab
ImageLab v1 Release: https://github.com/andreshungbz/imagelab/releases/tag/v1.0.0

IMAGELAB v2		
	Title	ImageLab v2 Report
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Oct 27, 2026

Table of Contents

Table of Contents **2**
Predictions **3**
Methodology **4**
Results **5**
Interpretation **8**
Limitations **9**
Recommendation **9**
Answered Questions **10**
Checkpoint Questions 10
Report Questions 11

IMAGELAB v2		
	Title	ImageLab v2 Report
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Oct 27, 2026

Predictions

All long polling predictions assume the default wait query parameter of 20 seconds. This is the timer that the handler holds before returning a 204 No Content response. These predictions also assume no artificial delays set for image processing.

Condition	Controlled Setup	Question	Short Poll Prediction	Long Poll Prediction
A Quiet job	Add 30 seconds of delay while processing, with no intermediate updates. Then finish actual processing.	How much observation traffic carries no new information?	Approximately 30 observation traffic requests will have no new information, given the 1s short polling rate.	Just 1 observation traffic request will carry no new information in its response (204 No Content).
B Real progress	Use actual image processing. Publish progress after each of the three variants is stored.	How quickly is completion discovered during ordinary use?	Within a maximum bound of 1s, depending on the moment the short poll fires. May skip versions	Completion is discovered very quickly within milliseconds. May skip versions.
C1 Slow updates	Before real processing, run a 10-second simulated phase with updates every 2 seconds.	What happens with occasional progress changes?	There will be approximately 2 short polls per simulated update, with 1 containing no new information. About 10 short polls.	There will be 1 long poll per simulated update. Each will contain new information. About 10 long polls.
C2 Fast updates	Use the same simulated phase with updates every 200 ms.	How do requests and observed versions change?	There will be about 10 short polls for the simulated phase, each containing new information.	There will be about 50 long polls for the simulated phase, each containing new information.
C3 Fast updates with latency	Repeat C2 with 200 ms additional round-trip latency using documented browser or network tooling.	What happens between successive requests on a slower path?	The approximate number of short pulls should remain the same at around 10. Even with 200ms latency and a 200ms simulated update rate (total 400ms), the polling 1s rate remains the rate-limiting factor.	There will be fewer long polls than in C2 because the latency causes another simulated update to fire before the next long poll. In the simulated phase, the number should be around 25 (10000ms/400ms).

IMAGELAB v2		
	Title	ImageLab v2 Report
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Oct 27, 2026

Methodology

Long-polling functionality was added to ImageLab using a timer-based notification approach, as shown in the example code provided. Following the example code, the ticker was set to fire every 200ms.

The experiments and measurements were conducted by running the browser and server on the same computer. A total of 30 measured runs were conducted for each of the 5 conditions (A, B, C1, C2, C3) and each of the 2 modes (short and long polling), with each run repeated 3 times. The same input image was used with each run starting with an empty worker queue and one observer. Between each condition, the starting mode was alternated.

Short polling was conducted at 1-second intervals, with a delay after each response to account for timing. Long polling immediately started its next request when the previous one completed.

For each run, organized by run ID, the raw browser network logs were exported to HTTP Archive (.har) files, and the raw server logs were copied to a .log file. Browser console logs were also recorded to a .log file. Logs were normalized between the backend and frontend to millisecond resolution and the America/Belize timezone (e.g., "2026-10-21T23:35:23.006-06:00"). Scheduling uncertainty is acknowledged in both the browser request queuing/processing and the execution of the server program. Recorded requests started after the page fully loaded.

From the raw measurements, the data was consolidated into a CSV summary, where the results were analyzed and refined into charts for request count by condition and completion detection by condition.

To better show the effects of latency, the 6th condition, C3-X, is used. It has the same conditions as C3, but with a 400ms latency set.

IMAGELAB v2		
	Title	ImageLab v2 Report
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Oct 27, 2026

Results

Refer to the following external links for the raw logs and files. The raw CSV summary is also provided in the Google Sheets form to make the results easier to view. It is not shown here because there are too many columns to reasonably fit. Figures 1 and 2 on the next page detail the charts derived from the results in Table 1 below.

Raw Logs/Files (GitHub):
https://github.com/andreshungbz/imagelab/tree/v2-long-polling/docs/v2.0.0/experiments
CSV Summary (Google Sheets): v2-csv-summary

Condition	Mode	Minimum Requests	Maximum Requests	Average Requests	Average Completion-Detection Delay
A	Short Polling	32.0	32.0	32.0	317.3
A	Long Polling	6.0	6.0	6.0	95.7
B	Short Polling	2.0	2.0	2.0	295.0
B	Long Polling	4.0	5.0	4.3	65.3
C1	Short Polling	12.0	12.0	12.0	328.3
C1	Long Polling	9.0	10.0	9.7	92.7
C2	Short Polling	12.0	12.0	12.0	336.0
C2	Long Polling	53.0	54.0	53.7	108.7
C3	Short Polling	12.0	12.0	12.0	729.7
C3	Long Polling	53.0	54.0	53.7	304.0
C3-X	Short Polling	12.0	12.0	12.0	1120.7
C3-X	Long Polling	27.0	27.0	27.0	563.0

Table 1: Results for each condition for each mode.

IMAGELAB v2		
	Title	ImageLab v2 Report
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Oct 27, 2026

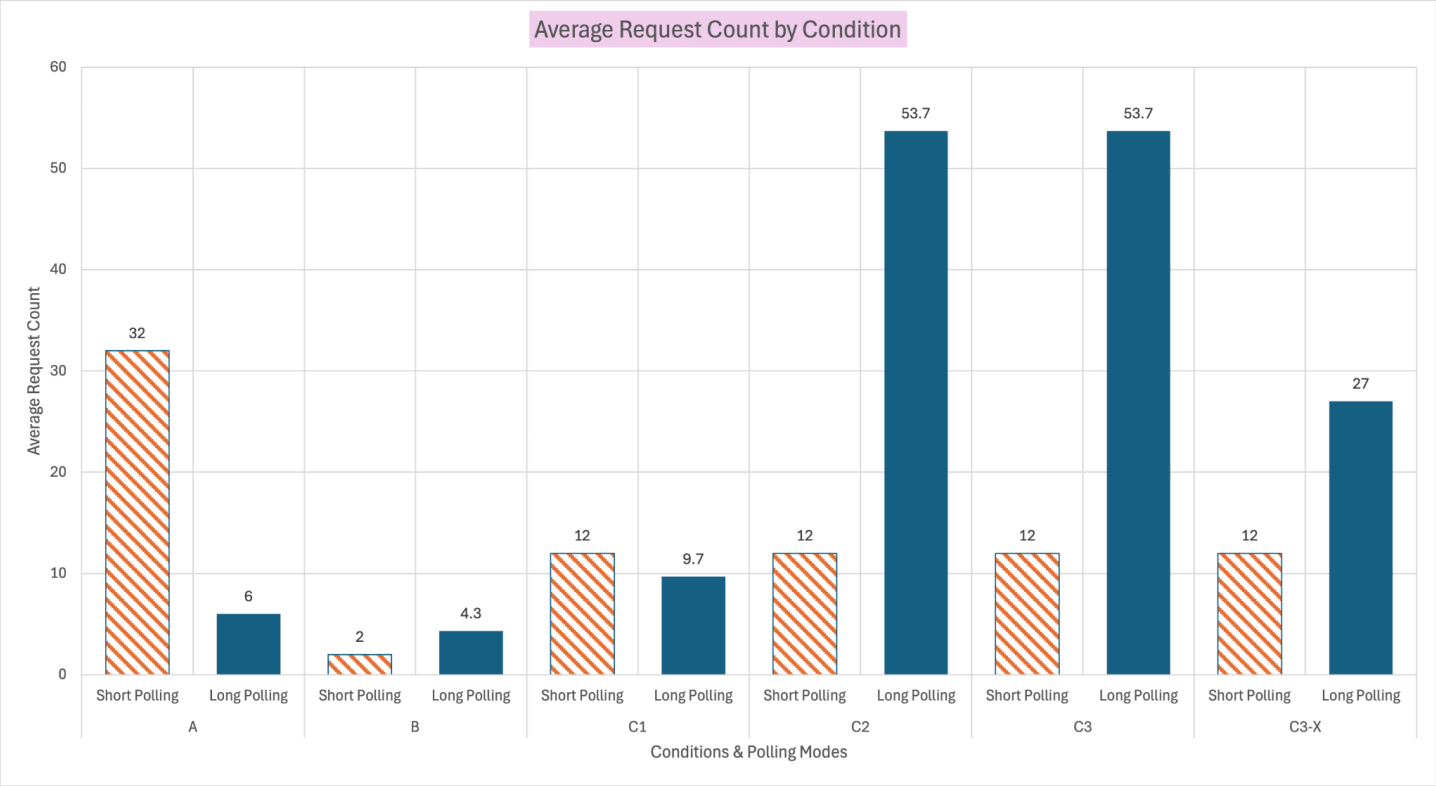


Figure 1: Chart for request count by condition.

IMAGELAB v2		
	Title	ImageLab v2 Report
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Oct 27, 2026

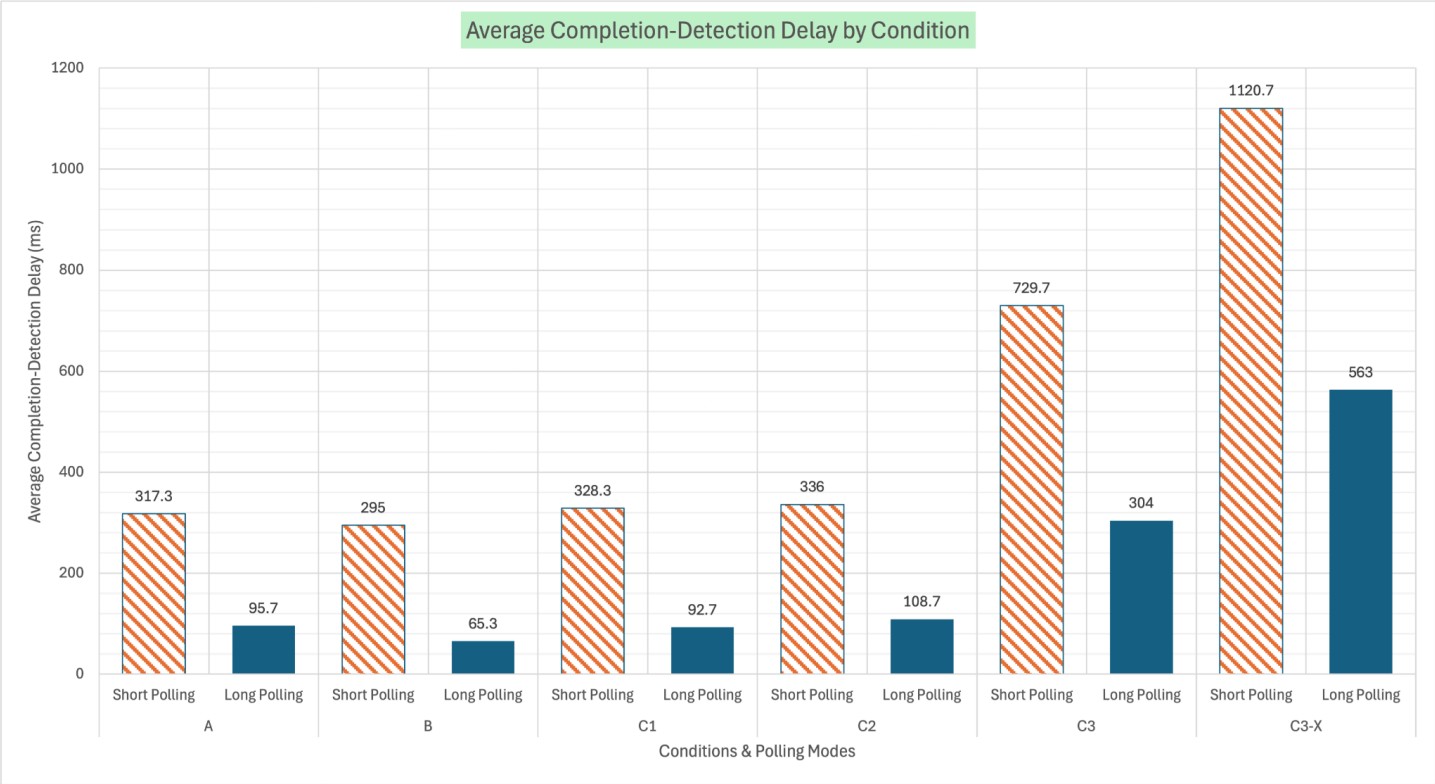


Figure 2: Chart for completion-detection delay by condition.

IMAGELAB v2		
	Title	ImageLab v2 Report
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Oct 27, 2026

Interpretation

For Condition A, the processing duration was consistent at approximately 30 seconds for both short and long polling. Because short polling uses a constant 1-second rate, versions 3 and 4 were not seen in the request logs. Long polling shows all versions in the log, given it sends another long poll immediately after receiving a new version. Long polling resulted in far fewer requests and a shorter detection delay than short polling, as expected, because short polling's constant rate doesn't hold the connection. The cost of long polling is that more observation database reads are made, since the backend ticker is set to 200ms. This is the only condition for which a 204 No Content response is seen, since the default 20s hold timer is exceeded. Our predictions matched these results.

For Condition B, versions observed for both short polling and long polling were sparse, as expected, since no artificial delays are set. While long polling sends a few more requests due to the immediate sending of the next poll, it maintains its shorter detection delay advantage over short polling. Our predictions matched these results. Notably, the detection delays for long polling were always shorter than those for short polling.

For Condition C1, both polling modes took around 10 seconds. The number of requests was about the same for both, though long polling had slightly fewer. Long polling still had the advantage of a shorter detection delay. Short polling had the advantage with fewer observation database reads. Our predictions matched these results.

For Condition C2, the higher simulated update rate led to long polling incurring more observation database reads. Short polling's count stayed consistent with Condition C1. The extra database reads are due to more frequent long polls resulting from the faster simulated update rate. Our predictions matched the results for request counts, with long polling having a higher count due to the fast updates.

For Condition C3, adding latency did not really change the request counts in Condition C2. It did, however, halve the observation database reads compared to Condition C2. Previously, long polling was predicted to have half the number of requests, but it turns out that the reason it's about the same is that the 200ms latency aligns with the 200ms backend ticker, so by the time the request and response have been sent, the client is able to send the next long poll nearly immediately. For this reason, Condition C3-X was conducted. With 400ms latency, halving the number of requests is incorrectly predicted for Condition C2. While short polling results in significantly longer detection delays for these conditions, it wins over long polling in the number of requests, which is consistently smaller for Conditions C1, C2, C3, and C3-X.

From these interpretations of the results, it can be seen that long polling is not necessarily better than short polling. Under certain conditions, long polling results in fewer requests. In others, it makes more. Consistently, long polling had a shorter detection delay than short polling across all conditions, although it also always had a higher count of observation database reads.

Limitations

- Timing issues due to the 200ms backend ticker for long polling or the 1s short polling rate may cause variations between runs under the same condition.
- The browser may have introduced delays due to packet queuing.
- The results were manually tested and recorded.
- Limitations pertaining to the single worker were not tested.
- Browser and server logs were normalized, but Go has larger precision than JavaScript.

Recommendation

Long polling is preferred when detection delay is of concern. It makes more database reads than short polling, especially when delays are involved. It is also more complex to implement compared to short polling. Where long polling fails, short polling can act as a fallback asynchronous API mechanism.

A future implementation of long polling can use a proper channel-based notification logic rather than the ticker-based approach done here. The other mechanisms excluded in the requirements (e.g., LISTEN/NOTIFY) could also remediate the issue of many database reads. Additional workers also remain a recommendation.

IMAGELAB v2		
	Title	ImageLab v2 Report
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Oct 27, 2026

Answered Questions

Checkpoint Questions

1. **What replaces the counter value?**

The job version replaces the counter value.

2. **What does a job version identify?**

The job version identifies changes to the job status and progressively completed image variants.

3. **What wakes a waiting handler?**

There are 3 things that wake a waiting handler (the get job status handler): A ticker (the main wake-up mechanism which causes the job to be read from the database), a timer (the holding time the handler waits before sending a 204 No Content back to the client's long poll), and a context cancellation request.

4. **Where is authoritative state stored?**

The authoritative state is stored in the PostgreSQL database. Job status and version are fields in the jobs table, where the handler/worker checks for the job.

5. **How do you prevent a change between checking and waiting from being overlooked?**

The ticker periodically polls for changes, but to prevent a change from being overlooked when the timer ends, a final authoritative check is performed at that time. If there was a change, a proper response is sent back to the client. Otherwise, a 204 No Content response is committed.

6. **Why does a notification never prove that processing succeeded?**

A notification pertains only to changes in the job record. That change can mean either a success (job completed) or a failure (job failed). Thus, a notification never proves that processing succeeded.

IMAGELAB v2		
	Title	ImageLab v2 Report
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Oct 27, 2026

Report Questions

1. **Where did long polling help, and where did it not?**

Long polling helped reduce the completion-detection delay. It sometimes helped reduce the number of requests overall, but under certain conditions, it greatly exacerbated the problem.

2. **Did image processing become faster, or did the browser learn the result sooner?**

Image processing did not become faster with long polling (consistent processing durations between short and long polling), but the mode did enable earlier learning of the result (a consistently shorter completion-detection delay compared to short polling).

3. **Can fewer HTTP requests hide repeated database queries?**

Yes. Repeated database queries may not be visible (intermediate versions are skipped due to timing), but the backend still executes them. In the case of Conditions C1 and C2, fewer long-polling requests resulted in fewer observation database reads. Each new request reads an initial snapshot.

4. **Why might intermediate versions be skipped without losing the final result?**

Intermediate versions might be skipped due to the combined timing effects of the 200ms ticker (long polling), the 1s polling rate (short polling), and any latency between the client and the server. Skipping them does not affect the final result, as long as the last poll queries the completed job.

5. **Would the answer change if every event had to be retained?**

Yes, the final result might be lost if every event had to be retained. At least in the current implementation, intermediate versions are not saved as separate records, so it is possible that, due to timing variations, the client would request a version that does not exist in the database.